Gaussian Processes III

Sparse Gaussian Processes

Edward Milsom

Bayesian Machine Learning • CM32024 • 2025–26 • Lecture 23

Department of
Computer Science

UNIVERSITY OF
BATH

In this Episode ...

- Gaussian processes are expensive for large datasets
- Sparse GPs summarise the data using **inducing points**
- Inducing points are introduced via **variational inference**
- Luckily, the maths is very similar to what we saw in GP classification VI, but the goal is very different.

Gaussian Process Recap

Gaussian Prior over Functions

- Let $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_N)]^T$ be the (unknown) function values at inputs $\{\mathbf{x}_n\}_{n=1}^N$
- A **Gaussian Process prior over functions** assumes that $\mathbf{f}$ is distributed according to a **multivariate Gaussian**:

$$\begin{pmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{pmatrix} \sim N\left(\begin{pmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_P) \end{pmatrix}, \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix} \right)$$

- $m(\cdot)$ is the **mean function** (often set to zero for simplicity).
- $k(\cdot, \cdot)$ is the **covariance function**, or **kernel**, which specifies **how correlated function values are** at different inputs.

Closed-form posterior (Gaussian noise observations)

- For observed data $\mathbf{X}, \mathbf{y}$, our **joint distribution** is

$$\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} \sim N\left(\begin{pmatrix} \mathbf{m} \\ \mathbf{m}_* \end{pmatrix}, \begin{pmatrix} \mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}} \mathbf{I} & \mathbf{K}(\mathbf{X}, \mathbf{X}_*) \\ \mathbf{K}(\mathbf{X}_*, \mathbf{X}) & \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) \end{pmatrix} \right)$$

  where $\mathbf{K} = \mathbf{K}(\mathbf{X}, \mathbf{X})$, $\mathbf{K}_* = \mathbf{K}(\mathbf{X}, \mathbf{X}_*)$, and $\mathbf{K}_{**} = \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*)$.
- Using the **partitioned Gaussian conditional formula**, we get:

$$\mathbf{f}_* | \mathbf{y} \sim N(\boldsymbol{\mu}_{*|y}, \boldsymbol{\Sigma}_{*|y})$$

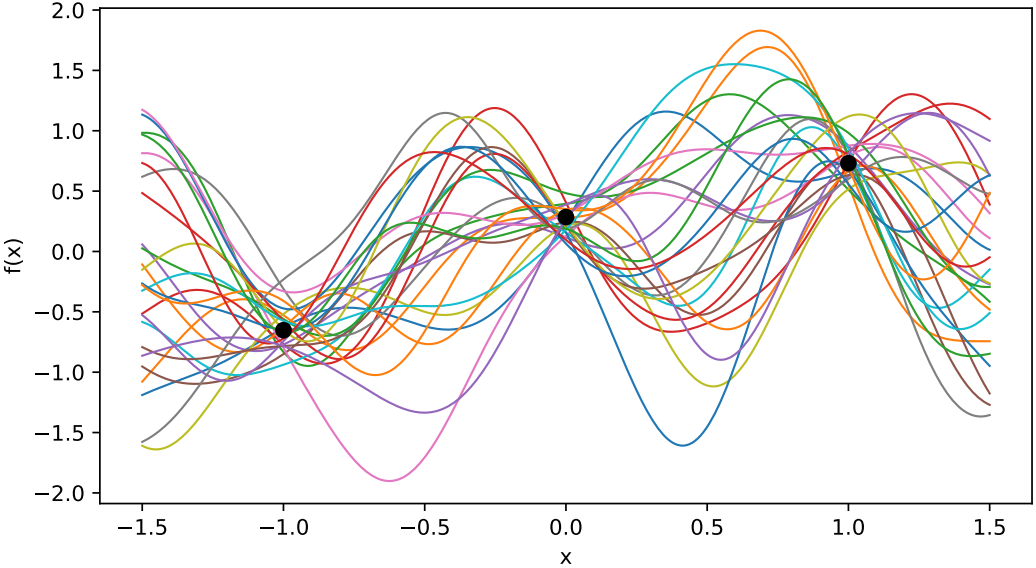
  where

$$\boldsymbol{\mu}_{*|y} = \mathbf{m}_* + \mathbf{K}(\mathbf{X}_*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}} \mathbf{I}]^{-1}(\mathbf{y} - \mathbf{m})$$

$$\boldsymbol{\Sigma}_{*|y} = \mathbf{K}(\mathbf{X}_*, \mathbf{X}_*) - \mathbf{K}(\mathbf{X}_*, \mathbf{X})[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}} \mathbf{I}]^{-1} \mathbf{K}(\mathbf{X}, \mathbf{X}_*)$$

- When using noiseless or Gaussian noise observations, and a GP prior, the **posterior is also a GP**!

GP posterior samples (Gaussian noise observations)



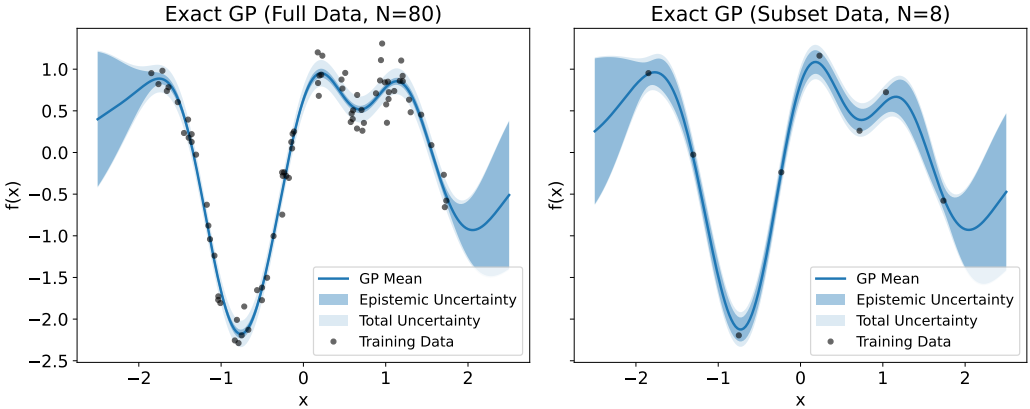
Computational Complexity

- GP prediction involves **inverting** the $N \times N$ kernel matrix $\mathbf{K}(\mathbf{X}, \mathbf{X})$ (plus noise):

$$[\mathbf{K}(\mathbf{X}, \mathbf{X}) + \sigma^2_{\text{noise}}\mathbf{I}]^{-1}$$

- For N datapoints, computing the kernel matrix requires $O(N^2)$ time and $O(N^2)$ **memory**. Inverting it requires $O(N^3)$ **time!**
- This cost was also present in the GP classification variational inference scheme (doubly so, because we also needed the log-determinant for the KL divergence, which is also $O(N^3)$).
- For large datasets (e.g. $N > 10,000$), this becomes **impractical**.

Subset of Training Data?



- Often a lot of data is **redundant** for learning the **shape** of the function.
- But simply subsetting the data can lead to **high uncertainty**, harming one of the key benefits of GPs. Can we improve on this idea?

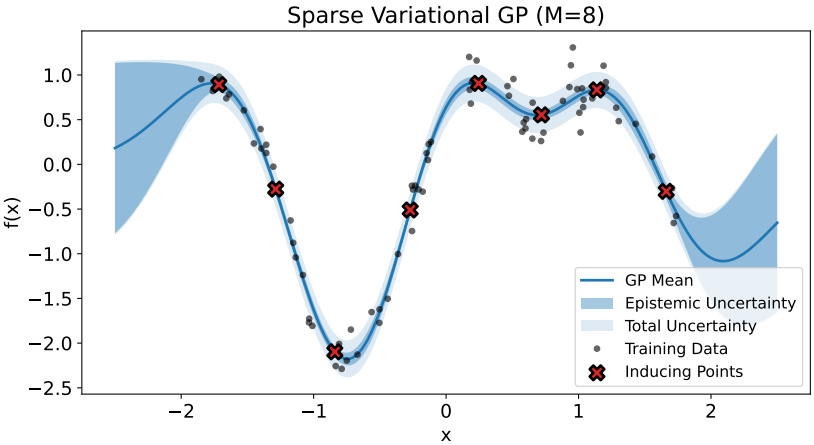
Sparse Variational Gaussian Processes

Inducing Points

- Sparse GPs use a small set of M pseudo-inputs, or **inducing points**, $\mathbf{X}_i \in \mathbb{R}^{M \times d}$ to summarise the data $\mathbf{X}_t$, with $M \ll N$.
- This means our kernel matrix inversions will be $O(M^3)$ instead of $O(N^3)$.
- The inducing points $\mathbf{X}_i$ may be a subset of the training inputs $\mathbf{X}_t$, or they may be **learned** during training.
- Rather than simply running the GP on the inducing points, we will use **variational inference** form an approximate posterior over the original GP that leverages inducing points to reduce computation.

The Goal

- **Low uncertainty** in high-data regions, but only $O(M^3)$ **inversions**.



Joint prior distribution with inducing points

- Let $\mathbf{X}_i \in \mathbb{R}^{M \times d}$ be our **inducing inputs**, and let $\mathbf{f}_i = f(\mathbf{X}_i) \in \mathbb{R}^M$ be the function values at those points (sometimes called **inducing outputs**).
- Furthermore, let $\mathbf{X}_t \in \mathbb{R}^{N \times d}$ be our **training data** inputs (with labels $\mathbf{y}_t$), and let $\mathbf{f}_t = f(\mathbf{X}_t) \in \mathbb{R}^N$ represent our function's values at those points.
- As before, we assume all function values are **jointly Gaussian**:

$$\begin{pmatrix} \mathbf{f}_i \\ \mathbf{f}_t \end{pmatrix} \sim N\left(\begin{pmatrix} \mathbf{0} \\ \mathbf{0} \end{pmatrix}, \begin{pmatrix} \mathbf{K}_{ii} & \mathbf{K}_{it} \\ \mathbf{K}_{ti} & \mathbf{K}_{tt} \end{pmatrix} \right)$$

  where $\mathbf{K}_{ii} = \mathbf{K}(\mathbf{X}_i, \mathbf{X}_i) \in \mathbb{R}^{M \times M}$, $\mathbf{K}_{it} = \mathbf{K}_{ti}^T = \mathbf{K}(\mathbf{X}_i, \mathbf{X}_t) \in \mathbb{R}^{M \times N}$, and $\mathbf{K}_{tt} = \mathbf{K}(\mathbf{X}_t, \mathbf{X}_t) \in \mathbb{R}^{N \times N}$.
- Note we have assumed a zero-mean GP prior for simplicity.

Inducing Point Approximate Posterior

- We form an approximate posterior over the joint values $\mathbf{f} = (\mathbf{f}_i, \mathbf{f}_t)$ as

$$q(\mathbf{f}; \boldsymbol{\theta}) = q(\mathbf{f}_i, \mathbf{f}_t; \boldsymbol{\theta}) = p(\mathbf{f}_t | \mathbf{f}_i) q(\mathbf{f}_i; \boldsymbol{\theta}), \qquad q(\mathbf{f}_i; \boldsymbol{\theta}) = N(\mathbf{f}_i; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_{ii})$$

  where $\boldsymbol{\theta} = \{\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_{ii}\}$ are our variational parameters to be optimised.
- Note the structure: we learn an approximate posterior $q(\mathbf{f}_i)$ over the inducing points, but use the exact conditional prior $p(\mathbf{f}_t | \mathbf{f}_i)$ (conditional Gaussians) to predict the training points given the inducing points.
- This is a slightly non-standard thing to do (usually we'd define $q(\mathbf{f}_t)$ directly), and results in a slightly looser bound on the evidence (see Titsias 2009 for details), but it **significantly simplifies computations**.

Fitting the Approximate Posterior

- We're doing variational inference, so we fit our approximate posterior by maximising the **evidence lower bound** (ELBO) with respect to $\boldsymbol{\theta}$:

$$\mathrm{ELBO}(\boldsymbol{\theta}) = \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f})] - \mathrm{KL}[q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f})]$$

- The way we structured $q(\mathbf{f};\boldsymbol{\theta})$ will allow simplifications for both terms.
- Let's look at the **expected log-likelihood** term first.

Expected Log-Likelihood

- The likelihood only depends on our training data, so

$$\mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f})] = \mathbb{E}_{q(\mathbf{f};\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f}_t)] = \mathbb{E}_{q(\mathbf{f}_t;\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f}_t)]$$

- As usual, if we know $q(\mathbf{f}_t;\boldsymbol{\theta})$, we can compute this expectation using **Monte Carlo sampling** (or analytically for Gaussian likelihood, though we leave this as an exercise):

$$\mathbb{E}_{q(\mathbf{f}_t;\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f}_t)] \approx \frac{1}{S}\sum_{s=1}^{S} \log p(\mathbf{y}_t|\mathbf{f}_t^{(s)}), \quad \mathbf{f}_t^{(s)} \sim q(\mathbf{f}_t;\boldsymbol{\theta})$$

- We obtain $q(\mathbf{f}_t;\boldsymbol{\theta})$ through a **Gaussian marginalisation**:

$$q(\mathbf{f}_t;\boldsymbol{\theta}) = \int p(\mathbf{f}_t|\mathbf{f}_i)q(\mathbf{f}_i;\boldsymbol{\theta})d\mathbf{f}_i$$

Expected Log-Likelihood — Gaussian Marginal

$$\mathbb{E}_{q(\mathbf{f}_t;\boldsymbol{\theta})}[\log p(\mathbf{y}_t|\mathbf{f}_t)] \approx \frac{1}{S}\sum_{s=1}^{S} \log p(\mathbf{y}_t|\mathbf{f}_t^{(s)}), \quad \mathbf{f}_t^{(s)} \sim q(\mathbf{f}_t;\boldsymbol{\theta})$$

- Since both $p(\mathbf{f}_t|\mathbf{f}_i)$ and $q(\mathbf{f}_i;\boldsymbol{\theta})$ are Gaussian, the marginal $q(\mathbf{f}_t;\boldsymbol{\theta})$ is also Gaussian:

$$q(\mathbf{f}_t;\boldsymbol{\theta}) = N(\mathbf{f}_t;\boldsymbol{\mu}_{t|i},\boldsymbol{\Sigma}_{tt|ii})$$

- This is essentially **identical** to the marginalisation we saw for GP classification VI (see **last lecture**), and so we have

$$\boldsymbol{\mu}_{t|i} = \mathbf{K}_{ti}\mathbf{K}_{ii}^{-1}\boldsymbol{\mu}_i$$

$$\boldsymbol{\Sigma}_{tt|ii} = \mathbf{K}_{tt} - \mathbf{K}_{ti}\mathbf{K}_{ii}^{-1}\mathbf{K}_{it} + \mathbf{K}_{ti}\mathbf{K}_{ii}^{-1}\boldsymbol{\Sigma}_{ii}\mathbf{K}_{ii}^{-1}\mathbf{K}_{it}$$

Expected Log-Likelihood — Minibatching

- Our likelihoods usually **factorise** over datapoints:

$$\log p(\mathbf{y}_t | \mathbf{f}_t) = \sum_{n=1}^{N} \log p(y_n | f(\mathbf{x}_n))$$

- Hence, our expected log-likelihood can be written as a **sum over datapoints**:

$$\mathbb{E}_{q(\mathbf{f}_t;\boldsymbol{\theta})}[\log p(\mathbf{y}_t | \mathbf{f}_t)] = \sum_{n=1}^{N} \mathbb{E}_{q(f(\mathbf{x}_n);\boldsymbol{\theta})}[\log p(y_n | f(\mathbf{x}_n))]$$

- For very large datasets, we can use **minibatching** to estimate this, avoiding the full $\mathbf{K}_{tt}$ computation:

$$\mathbb{E}_{q(\mathbf{f}_t;\boldsymbol{\theta})}[\log p(\mathbf{y}_t | \mathbf{f}_t)] \approx \frac{N}{B} \sum_{b=1}^{B} \mathbb{E}_{q(f(\mathbf{x}_b);\boldsymbol{\theta})}[\log p(y_b | f(\mathbf{x}_b))]$$

Expected Log-Likelihood — Diagonal

- Previous slide: we only need the **marginal** $q(f(\mathbf{x}_n); \boldsymbol{\theta})$ for each datapoint, rather than the full joint $q(\mathbf{f}_t; \boldsymbol{\theta}) = N(\mathbf{f}_t; \boldsymbol{\mu}_{t|i}, \boldsymbol{\Sigma}_{tt|ii})$.
- This means we only need the **diagonal** of $\boldsymbol{\Sigma}_{tt|ii}$ (no covariances):

$$\mathrm{diag}(\boldsymbol{\Sigma}_{tt|ii}) = \mathrm{diag}(\mathbf{K}_{tt}) - \mathrm{diag}(\mathbf{K}_{ti}\mathbf{K}_{ii}^{-1}\mathbf{K}_{it}) + \mathrm{diag}(\mathbf{K}_{ti}\mathbf{K}_{ii}^{-1}\boldsymbol{\Sigma}_{ii}\mathbf{K}_{ii}^{-1}\mathbf{K}_{it})$$

- This reduces the cost associated with computing $\mathbf{K}_{tt}$ from $O(B^2)$ to $O(B)$.
- The covariances between training data are implicitly captured by the inducing points $q(\mathbf{f}_i; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_{ii})$ (here $\boldsymbol{\Sigma}_{ii}$ is a full matrix).

KL Divergence Term

- In the KL divergence, the conditional prior terms cancel, giving a KL divergence over only inducing points:

$$
\begin{aligned}
&\mathrm{KL}[q(\mathbf{f};\boldsymbol{\theta})||p(\mathbf{f})] \\
&= \int \int p(\mathbf{f}_t|\mathbf{f}_i) q(\mathbf{f}_i;\boldsymbol{\theta}) \log \frac{p(\mathbf{f}_t|\mathbf{f}_i) q(\mathbf{f}_i;\boldsymbol{\theta})}{p(\mathbf{f}_t|\mathbf{f}_i) p(\mathbf{f}_i)} d\mathbf{f}_t d\mathbf{f}_i \\
&= \int q(\mathbf{f}_i;\boldsymbol{\theta}) \log \frac{q(\mathbf{f}_i;\boldsymbol{\theta})}{p(\mathbf{f}_i)} \left(\int p(\mathbf{f}_t|\mathbf{f}_i) d\mathbf{f}_t \right) d\mathbf{f}_i \\
&= \int q(\mathbf{f}_i;\boldsymbol{\theta}) \log \frac{q(\mathbf{f}_i;\boldsymbol{\theta})}{p(\mathbf{f}_i)} d\mathbf{f}_i = \mathrm{KL}[q(\mathbf{f}_i;\boldsymbol{\theta})||p(\mathbf{f}_i)] \\
&= \mathrm{KL}[N(\mathbf{f}_i;\boldsymbol{\mu}_i,\boldsymbol{\Sigma}_{ii})||N(\mathbf{f}_i;\mathbf{0},\mathbf{K}_{ii})] = \frac{1}{2}\left(\mathrm{tr}(\mathbf{K}_{ii}^{-1}\boldsymbol{\Sigma}_{ii}) + \boldsymbol{\mu}_i^T \mathbf{K}_{ii}^{-1} \boldsymbol{\mu}_i - M + \log \frac{|\mathbf{K}_{ii}|}{|\boldsymbol{\Sigma}_{ii}|} \right)
\end{aligned}
$$

ELBO — Summary

- We learn an approximate posterior $q(\mathbf{f}_i; \boldsymbol{\theta}) = N(\mathbf{f}_i; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_{ii})$ at the inducing points.
- The expected log-likelihood term is approximated in a similar way to GP classification VI (combination of **Monte Carlo sampling** and **Gaussian marginalisation**)
- We only need the diagonal of $\boldsymbol{\Sigma}_{tt|ii}$, and we can use **minibatching** to avoid computing the entire dataset at once if required.
- The **KL divergence term only applies to the inducing points**, meaning we only need to invert / log-det the $M \times M$ matrix $\mathbf{K}_{ii}$.

Prediction at Unseen Test Points

- Since the inducing points take the place of the training points in the model, predictions at unseen points $\mathbf{X}_*$ are **exactly the same** as for the training points (when we computed the expected log-likelihood):

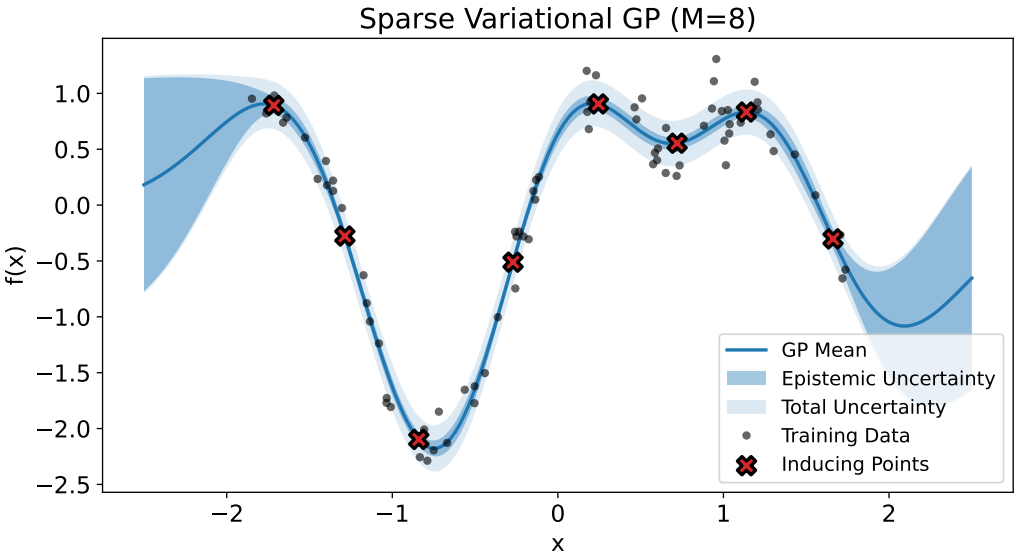
$$q(\mathbf{f}_*; \boldsymbol{\theta}) = \int p(\mathbf{f}_* | \mathbf{f}_\mathrm{i}) q(\mathbf{f}_\mathrm{i}; \boldsymbol{\theta}) d\mathbf{f}_\mathrm{i} = N(\mathbf{f}_*; \boldsymbol{\mu}_{*|i}, \boldsymbol{\Sigma}_{**|ii}).$$

- Here, $\boldsymbol{\mu}_{*|ii}, \boldsymbol{\Sigma}_{**|ii}$ are exactly the same as $\boldsymbol{\mu}_{\mathrm{t}|\mathrm{i}}, \boldsymbol{\Sigma}_{\mathrm{tt}|\mathrm{ii}}$ (see Slide 13), but with the kernel matrices computed on test points instead of training points.
- Remember, we optimised $\boldsymbol{\theta} = \{\boldsymbol{\mu}_\mathrm{i}, \boldsymbol{\Sigma}_\mathrm{ii}\}$ using the ELBO on the training data, thereby **summarising** the information.

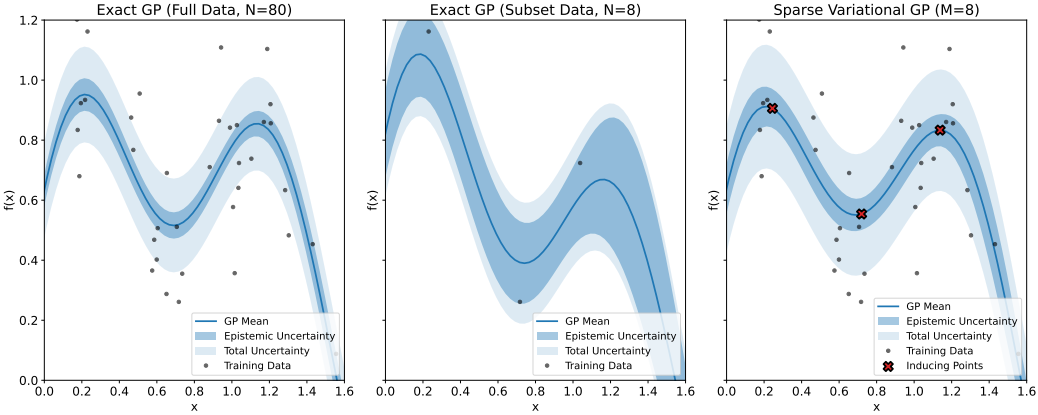
Choosing Inducing Inputs

- Our approximate posterior $q(\mathbf{f}_i; \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_{ii})$ learns the function values (and covariances) at inducing inputs $\mathbf{X}_i$. How should we choose $\mathbf{X}_i$?
- Completely randomly (not recommended without further tuning).
- **Random subset** of training inputs.
- More sophisticated methods e.g. **k-means clustering** on training inputs.
- **Optimise** them by **maximising the ELBO** (initialising with one of the above methods).
- Optimisation is sometimes difficult, so try different techniques.

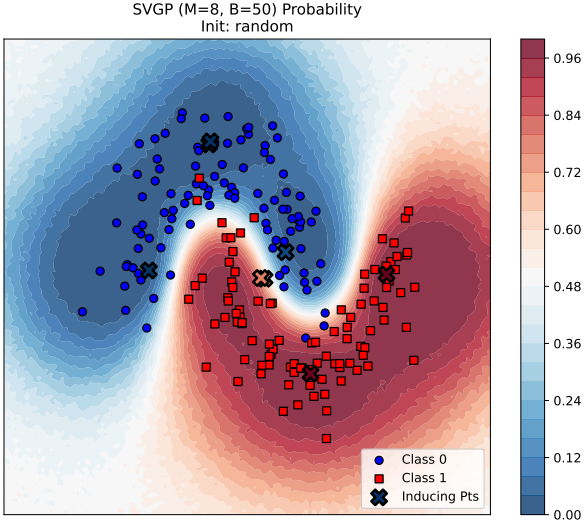
The Result


Sparse Variational GP (M=8)
f(x)
x
1.0
0.5
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−2
−1
0
1
2
GP Mean
Epistemic Uncertainty
Total Uncertainty
Training Data
Inducing Points

Sparse GPs give Lower Uncertainty than Subsampling



Classification Example



Conclusion

Takeaways

- Full-rank GPs are **too expensive** for large datasets ($O(N^3)$ time, $O(N^2)$ memory).
- Datasets can often be summarised using a small set of $M \ll N$ **inducing points**.
- Sparse GPs incorporate inducing points into a **variational approximate posterior**, which summarises the information from our training data, whilst keeping computations cheap ($O(M^3)$ **time,** $O(M^2)$ **memory**).
- Many equations are similar to GP classification VI, but the goal is different (but once you've implemented a sparse GP, switching between regression and classification is usually straightforward!).

Next Lecture

- **Bayesian Optimisation**
- A key application of Gaussian processes
- How do we maximise a function that is expensive to evaluate?
- The **flexibility** and **uncertainty quantification** of GPs are essential here.

References and Further Reading

References and Further Reading

- Leibfried et. al. 2022 *A Tutorial on Sparse Gaussian Processes and Variational Inference*. `https://arxiv.org/abs/2012.13962`
- Alexander Graeme de Garis Matthews *Scalable Gaussian process inference using variational methods*. University of Cambridge, 2016. `https://www.repository.cam.ac.uk/items/a2c7e9a7-d7bb-40c0-9285-8e0f487869e1`
- Hensman et. al. 2015 *Scalable Variational Gaussian Process Classification*. `https://arxiv.org/abs/1411.2005`